

Lab 14 - Smooth Filtering

June 7, 2025

1 Theory

Smoothing filters are used in image processing to reduce noise and minor variations in images. The most common smoothing filter is the mean filter, which replaces each pixel value in an image with the average value of its neighbors, effectively blurring the image and reducing detail. Mathematically, for a 3×3 mean filter, the output pixel value $g(x, y)$ is calculated as:

$$g(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 f(x + i, y + j)$$

where $f(x, y)$ is the original pixel value at position (x, y) .

A weighted smoothing filter, such as the Gaussian filter, assigns different weights to neighboring pixels, giving more importance to the central pixels. For example, a 3×3 weighted filter can be represented as:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

This approach preserves edges better while still reducing noise.

```
[1]: import numpy as np
      from PIL import Image
      import matplotlib.pyplot as plt
```

```
[2]: def smooth_filter(input_image):
      img = input_image.resize((400,400), Image.Resampling.LANCZOS)

      fig = plt.figure()
      fig.set_figheight(20)
      fig.set_figwidth(20)

      #plotting original image
      fig.add_subplot(1,2,1)
      plt.imshow(img, cmap='gray')
      plt.title('original')
```

```

# convert to numpy array
numpy_image = np.array(img)
# array for padding
array_b = np.zeros((402,402))

# to pad initial array with zeros in all side
array_b[1:401,1:401] = numpy_image

#defining filter
filter_array = np.array([[1/9,1/9,1/9],
                        [1/9,1/9,1/9],
                        [1/9,1/9,1/9]])

#creating empty list
lst = []

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

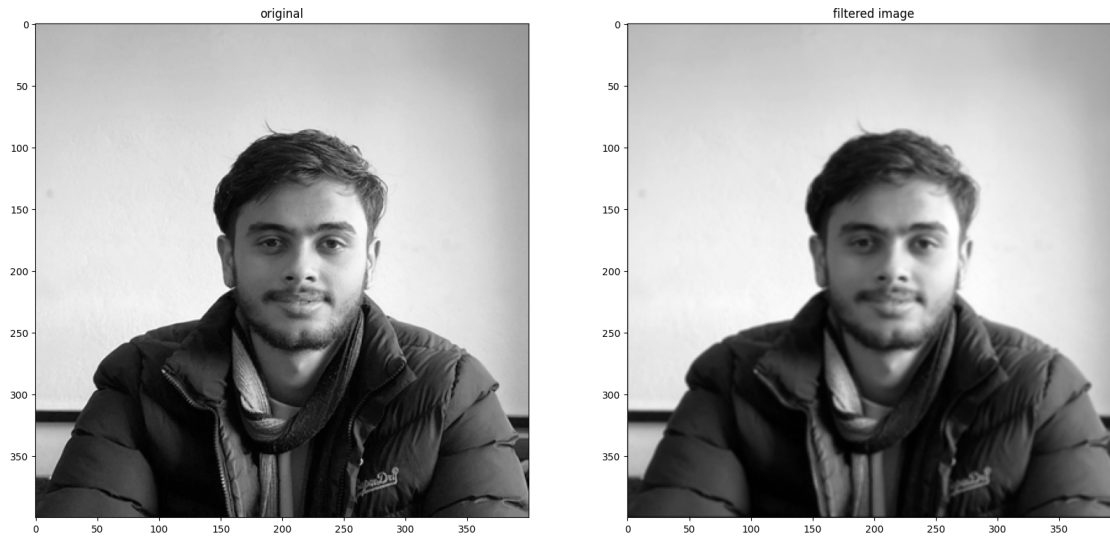
#plotting filtered image
fig.add_subplot(1,2,2)
plt.imshow(final_image, cmap='gray')
plt.title('filtered image')

```

```

[3]: # reading image and converting to gray scale
img = Image.open('img/samir.jpg').convert('L')
# Calling smooth function
smooth_filter(img)

```



```
[6]: def weighted_smooth_filter(input_image):
    img = input_image.resize((400,400), Image.Resampling.LANCZOS)

    fig = plt.figure()
    fig.set_figheight(20)
    fig.set_figwidth(20)

    #plotting original image
    fig.add_subplot(1,2,1)
    plt.imshow(img, cmap='gray')
    plt.title('original')

    # convert to numpy array
    numpy_image = np.array(img)
    # array for padding
    array_b = np.zeros((402,402))

    # to pad initial array with zeros in all side
    array_b[1:401,1:401] = numpy_image

    #defining filter
    filter_array = np.array([[1/16,2/16,1/16],
                             [2/16,4/16,2/16],
                             [1/16,2/16,1/16]])

    #creating empty list
    lst = []
```

```

for i in range(400):
    for j in range(400):
        #extracting part of array equal to filter size
        array_c = array_b[i:(3+i),j:(3+j)]

        #applying filter
        array_mul = np.multiply(filter_array,array_c)
        array_sum = np.sum(array_mul)

        # putting calculated value in list
        lst.append(array_sum)

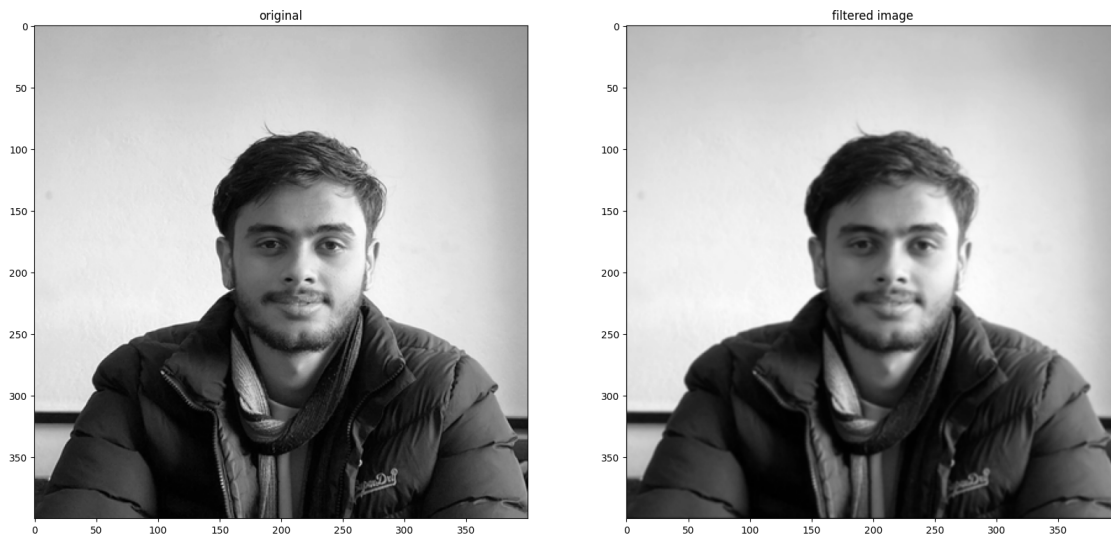
# resizing lst to shape of original array
final_array = np.resize(lst,(400,400))

final_image = Image.fromarray(final_array)
final_image= final_image.convert("L")

#plotting filtered image
fig.add_subplot(1,2,2)
plt.imshow(final_image, cmap='gray')
plt.title('filtered image')

```

```
[7]: weighted_smooth_filter(img)
```



2 Conclusion

Smoothing filters are essential tools in image processing for noise reduction and image enhancement. The mean filter provides uniform smoothing but may blur edges, while weighted filters like the Gaussian filter offer better edge preservation. By applying these filters, images become less noisy and more visually appealing, facilitating further analysis and processing.